

«Talento Tech»

React JS

Clase 08



Clase N° 8 | Introducción a Context API

Índice

Clase N° 8 | Introducción a Context API

Introducción a la problemática del Prop Drilling

¿Qué es la Context API de React?

Creación de nuestro componente "Carrito"

Creamos nuestro "CartContext"

Crear el archivo CartContext.jsx

Envolver la aplicación con el Provider

Haciendo que funcional el botón "Agregar producto"

Renderizado de la cantidad y los productos en el carrito

Paso 1: Modificar header.jsx (donde tenemos nuestro navbar)

Paso 2: Modificar Cart.jsx para mostrar los productos

Despliegue de la aplicación en Vercel o Netlify

Desplegar en Vercel

Desplegar en Netlify

Ejercicio práctico:

Cargar productos con fetch en ItemListContainer

Consignas Pre-Entrega de Proyecto Final

Requerimientos del Proyecto:

Preguntas para Reflexionar:

Próximos Pasos:

Objetivos de la Clase:

- Comprender las limitaciones del manejo de estado local en aplicaciones complejas.
- Aprender a identificar y solucionar el *prop drilling*.
- Implementar la Context API para crear un estado global y accesible desde cualquier componente.
- Centralizar la lógica de nuestro carrito de compras para que sea más mantenible y escalable.
- Poder subir una aplicación React en Vercel y Netlify.

Introducción a la problemática del *Prop Drilling*

Clases anteriores implementamos la lógica para agregar productos a un "carrito". Lo hicimos utilizando un estado local (useState) dentro de nuestro componente contenedor de productos. Esto funcionaba pero tenía una limitación fundamental: **ese estado vivía y moría dentro de ese único componente**.

Ahora que nuestra aplicación creció gracias al enrutamiento, nos enfrentamos a un nuevo desafío:

- El componente NavBar.jsx necesita saber cuántos productos hay en el carrito para mostrar un contador.
- Necesitaremos una página /carrito que muestre todos los productos agregados.

Imaginemos esta estructura:

```
<App>
  <EjemploItemListCont>
    <EjemploItem>
      <EjemploItemDetail>
        <BotonDeCompra />
      </EjemploItemDetail>
    </EjemploItem>
  </EjemploItemListCont>
</App>
```

La única forma que conocemos hasta ahora para comunicar estos componentes es **pasando props de padres a hijos**. Si el estado del carrito viviera en App.jsx, tendríamos que pasarlo a través de EjemploItemListCont.jsx, luego a EjemploItem.jsx, y así sucesivamente. Este proceso de "perforar" nuestro árbol de componentes con props que los componentes intermedios no necesitan se conoce como **Prop Drilling**.

El Prop Drilling es una mala práctica porque:

- **Hace el código difícil de mantener:** Si un componente en el medio deja de pasar una prop, todo lo que esté por debajo se rompe.
- **Genera acoplamiento:** Los componentes se vuelven dependientes de otros solo para pasar información.
- **Es verboso e ineficiente.**

¿Qué es la Context API de React?

Para resolver este inconveniente, React nos provee la **Context API**. Esta API nos permite crear un "contexto", que es una suerte de almacén de datos global al que cualquier componente, sin importar su nivel de anidamiento, puede suscribirse para leer o modificar su información, evitando así el *prop drilling*.

El funcionamiento de la Context API se basa en dos elementos principales:

- **Provider (Proveedor):** Es un componente que "envuelve" a una parte de nuestro árbol de componentes. Todo lo que esté dentro de este Provider tendrá acceso a la información del contexto.
- **Consumer (Consumidor):** Es la forma en que los componentes acceden a la información del contexto. Modernamente, se utiliza el hook `useContext` para este fin, ya que es más declarativo y simple.

Esta API nos permite compartir estado de forma implícita, evitando por completo el Prop Drilling.

Creación de nuestro componente "Carrito"

Antes de centralizar la lógica, necesitamos una vista para el carrito.

1. Dentro de `src/componentes`, creá una nueva carpeta `Cart` y dentro, un archivo `Cart.jsx`.
2. Por el momento vamos a hacer que muestre un mensaje sencillo. Podés escribirle lo que quieras. Te dejo un ejemplo:

```
// src/componentes/Cart/Cart.jsx
import React from 'react';
const Cart = () => {
  // Por ahora, este componente solo mostrará un mensaje.
  // Más adelante, consumirá los datos de nuestro contexto.
  return (
    <div>
      <h1>Carrito de Compras</h1>
      <p>Aquí se mostrarán los productos que agregaste.</p>
    </div>
  );
};

export default Cart;
```

Como vimos en la clase anterior, vamos a agregar la nueva ruta en nuestro `App.jsx`

```
// En src/App.jsx
// Otras importaciones
import Layout from './componentes/layout/Layout';
import ProductoDetalle from './componentes/Productos/ProductoDetalle';
import Cart from './componentes/Cart/cart'; // Importamos el componente creado
function App() {
  return (
    <Routes>
      <Route element={<Layout />}
        {/* Otras rutas...*/}
        <Route path="/carrito" element={<Cart />} />
      </Route>
    </Routes>
  );
}

export default App;
```

Proba yendo a **/carrito**. Si ves el contenido hecho en Cart.jsx entonces vamos bien. Si no llegas a visualizar el contenido, consulta con tu instructor.

Creamos nuestro "CartContext"

Vamos a unir todo creando el contexto que englobe la información necesaria. No solo crearemos el contexto, sino que lo dejaremos listo para ser implementado en nuestros componentes existentes.

Crear el archivo CartContext.jsx

Primero, creamos la estructura que ya conocemos, pero esta vez la pensaremos como el "cerebro" central de nuestro carrito. En la raíz de src, crea una nueva carpeta llamada context. Dentro, crea el archivo CartContext.jsx con el siguiente código explicado paso a paso:

1. Importaciones necesarias

```
import React, { useState, useContext, createContext } from 'react';
```

- useState → para guardar y actualizar el carrito.
- useContext → para poder consumir el contexto desde los componentes.
- createContext → para crear el contexto.

2. Creación del contexto

```
export const CartContext = createContext();
```

Aquí estamos creando el contexto en sí. Pensá que el contexto es como una “caja” donde vamos a guardar y compartir datos (en este caso, el carrito).

3. Custom Hook useCart

```
export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart debe ser usado dentro de un CartProvider');
  }
  return context;
};
```

- Crearemos esta hook personalizada porque nos facilita el consumo del contexto en cualquier componente.
- En lugar de escribir useContext(CartContext) cada vez, simplemente usaremos useCart().
- Si alguien intenta usarlo fuera del proveedor (CartProvider), lanzará un error claro.

4. El CartProvider (nuestro cerebro 🧠)

```
export const CartProvider = ({ children }) => {
  const [cart, setCart] = useState([]);
```

- CartProvider es un componente especial que envolverá a toda tu app.
- Dentro, usamos un useState([]) → el carrito arranca vacío.

5. Creamos las funciones que manejan el carrito

a. addToCart

```
const addToCart = (product, quantity) => {
  const itemInCart = cart.find(item => item.id === product.id);
  if (itemInCart) {
    const updatedCart = cart.map(item =>
      item.id === product.id
        ? { ...item, quantity: item.quantity + quantity }
        : item
    );
    setCart(updatedCart);
  } else {
    setCart(prevCart => [...prevCart, { ...product, quantity }]);
  }
};
```

- Busca si el producto ya está en el carrito.
- Si está → suma la cantidad.
- Si no está → lo agrega como nuevo.

b. clearCart

```
const clearCart = () => {
  setCart([]);
};
```

- Vacía el carrito

c. getCartQuantity

```
const getCartQuantity = () => {
  return cart.reduce((acc, item) => acc + item.quantity, 0);};
```

- Recorre el carrito y suma todas las cantidades.
- Sirve para mostrar, por ejemplo, el numerito de productos en un ícono .

d. getCartTotal

```
const getCartTotal = () => {
  return cart.reduce((acc, item) => acc + item.precio * item.quantity,
0);
};
```

- Calcula el precio total de todos los productos en el carrito.

6. Retorno del CartProvider

```
return (
  <CartContext.Provider value={{ cart, addToCart, clearCart,
getCartQuantity, getCartTotal }}>
    {children}
  </CartContext.Provider>
);
};
```

- **CartContext.Provider** es el que realmente comparte la información.
- El value={{ ... }} contiene todo lo que cualquier componente va a poder usar:
 - cart → el estado del carrito.
 - addToCart, clearCart, getCartQuantity, getCartTotal → las funciones que controlan el carrito.
- {children} → asegura que todo lo que envuelva CartProvider (tu app o un grupo de componentes) pueda acceder al carrito.

Código completo en la siguiente hoja

Código completo:

```
// src/context/CartContext.jsx
import React, { useState, useContext, createContext } from 'react';

export const CartContext = createContext();

export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) {
    throw new Error('useCart debe ser usado dentro de un CartProvider');
  }
  return context;
};

export const CartProvider = ({ children }) => {
  const [cart, setCart] = useState([]);

  const addToCart = (product, quantity) => {
    const itemInCart = cart.find(item => item.id === product.id);
    if (itemInCart) {
      const updatedCart = cart.map(item =>
        item.id === product.id
          ? { ...item, quantity: item.quantity + quantity }
          : item
      );
      setCart(updatedCart);
    } else {
      setCart(prevCart => [...prevCart, { ...product, quantity }]);
    }
  };

  const clearCart = () => {
    setCart([]);
  };

  const getCartQuantity = () => {
    return cart.reduce((acc, item) => acc + item.quantity, 0);
  };

  const getCartTotal = () => {
    return cart.reduce((acc, item) => acc + item.precio * item.quantity, 0);
  };

  return (
    <CartContext.Provider value={{ cart, addToCart, clearCart,
    getCartQuantity, getCartTotal }}>
      {children}
    </CartContext.Provider>
  );
};
```


Envolver la aplicación con el Provider

Este es un paso muy importante y que no puede faltar. Hacemos que el "cerebro" esté disponible para toda la App.

En `src/main.jsx` importamos `CartProvider` y envolvemos el componente `<App />`

```
// src/main.jsx
// Otras importaciones
import { BrowserRouter } from 'react-router-dom'; // Importamos BrowserRouter
import { CartProvider } from '../context/CartContext.jsx';
createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <CartProvider> { /* Envolvemos la App */}
    <App />
  </CartProvider>
</BrowserRouter>
)
```

Haciendo que funcional el botón "Agregar producto"

Vamos a modificar el componente que muestra los productos para que utilice la función `addToCart` de nuestro contexto. En esta oportunidad vamos a mostrar únicamente los productos hardcodeados.

1. Modificar el componente `Item.jsx` (o donde quieras agregar el botón):

```
// En /componentes/Item/Item.jsx
import { Link } from 'react-router-dom';
import { useCart } from '../../context/CartContext';
import { useState } from 'react';

export function Item({ id, nombre, precio, stock, imagen }) {
  // Creamos el objeto producto a partir de las props
  const producto = { id, nombre, precio, stock, imagen };

  const [cantidad, setCantidad] = useState(0);

  // Mantenemos la función incrementar, decrementar y la lógica de "favorito"

  // Lógica del Carrito
  const { addToCart } = useCart(); // Traemos la función del contexto

  const handleAddToCart = () => {
    addToCart(producto, cantidad);
    alert(`Agregaste ${cantidad} unidades de ${nombre} al carrito.`);
  };

  return (
    <div className="card-producto">
```

```

        <img src={producto.imagen} alt={producto.nombre} width={100}
height={100} />
        <h3>{producto.nombre}</h3>
        <p>${producto.precio}</p>
        {/* Incorporamos el detalle del stock */}
        <p>Stock disponible: {stock}</p>
        <Link to={`/${producto}/${producto.id}`}>Ver detalle</Link>
        {/* Mantenemos el favorito y los botones como estaban */}
        <button onClick={handleAddToCart}>
            Agregar {cantidad} al carrito
        </button>
    </div>
    );
}
export default Item;

```

Renderizado de la cantidad y los productos en el carrito

Ahora que los productos se pueden agregar, necesitamos verlos reflejados en nuestra barra de navegación y en la página /carrito.

Paso 1: Modificar header.jsx (donde tenemos nuestro navbar)

```

import styles from './Header.module.css';
import { Link } from 'react-router-dom'; // 1. Importamos Link
// 1. Importamos nuestro custom Hook
import { useCart } from '../context/CartContext';

function Header() {
    // 2. Usamos el hook para acceder a la función
    const { getCartQuantity } = useCart();
    const totalItems = getCartQuantity();

    return (
        <header className={styles.header}>
            <nav>
                <ul>
                    // Otros items
                    <li><Link to="/carrito">Carrito 🛒 {totalItems > 0 &&
<span>{totalItems}</span></Link></li>
                </ul>
            </nav>
        </header>
    );
};

export default Header;

```

Paso 2: Modificar Cart.jsx para mostrar los productos

Finalmente, hacemos que la vista del carrito sea dinámica y muestre lo que realmente contiene nuestro estado global.

```

// src/components/Cart/Cart.jsx
import React from 'react';
import { useCart } from '../../context/CartContext'; // 1. Importamos el hook

const Cart = () => {
  // 2. Obtenemos el estado 'cart' y las funciones que necesitamos del
  contexto
  const { cart, clearCart, getCartTotal } = useCart();

  // Si el carrito está vacío, mostramos un mensaje
  if (cart.length === 0) {
    return (
      <div>
        <h1>El carrito está vacío</h1>
        <p>Agrega productos para continuar la compra.</p>
      </div>
    );
  }

  // Si hay productos, los mostramos
  return (
    <div>
      <h1>Carrito de Compras</h1>
      {cart.map(item => (
        <div key={item.id} className="cart-item">
          <h4>{item.nombre}</h4>
          <p>Cantidad: {item.quantity}</p>
          <p>Precio unitario: ${item.precio}</p>
          <p>Subtotal: ${item.precio * item.quantity}</p>
        </div>
      ))}
      <hr />
      <h3>Total a pagar: ${getCartTotal()}</h3>
      <button onClick={clearCart}>Vaciar Carrito</button>
    </div>
  );
};

export default Cart;

```

Despliegue de la aplicación en Vercel o Netlify

Hemos llegado al punto en que nuestra aplicación, que hasta ahora ha vivido en nuestro entorno de desarrollo local, está lista para ser presentada al mundo. El despliegue, o *deploy*, es el proceso de tomar el código de nuestra aplicación y publicarlo en un servidor web para que cualquier usuario con acceso a internet pueda utilizarla



Configuración del Proyecto para Despliegue

Antes de iniciar el despliegue, es imperativo realizar una serie de verificaciones para garantizar un proceso sin contratiempos. Este es un paso profesional que nos distingue y asegura la calidad del producto final.

Pasos previos fundamentales:

-  **Revisar el funcionamiento local:** Ejecutar **npm run dev** por última vez y navegar por toda la aplicación. Probar el CRUD de productos, la autenticación, el carrito de compras y la responsividad.
-  **Verificar dependencias:** Asegurarse de que el archivo package.json contenga todas las dependencias necesarias. Una buena práctica es ejecutar **npm install** para confirmar que el proyecto se construye sin errores a partir de sus dependencias declaradas.
-  **Variables de entorno:** Si nuestra aplicación utiliza variables de entorno (por ejemplo, para las credenciales de ImgBB en un archivo .env), debemos saber que estos archivos no se suben a los repositorios de Git por seguridad. Las plataformas de despliegue nos ofrecerán una interfaz para configurarlas de forma segura.
-  **Control de versiones:** Realizar un commit en Git con todos los cambios finales. Es una buena práctica etiquetar este commit como la versión de despliegue (ej: git commit -m "v1.0 - P para despliegue")

Desplegar en Vercel

Vercel es una plataforma optimizada para el despliegue de aplicaciones de *frontend*, especialmente aquellas construidas con Next.js (el framework de los mismos creadores) y, por supuesto, React. Su integración con GitHub es excepcional.



Para hacer una subida limpia y evitar errores vamos a limpiar nuestro proyecto siguiendo estos pasos:

1. Asegurate que las importaciones estén bien nombradas, teniendo en cuenta que es sensible a mayúsculas y minúsculas.
2. Borrá la carpeta `node_modules`
3. Borrá el archivo `package-lock.json`. **NO EL `package.json`.**
4. Probamos el desarrollo con **`npm run build`**.

Si no hay errores, pasamos al Deploy en Vercel

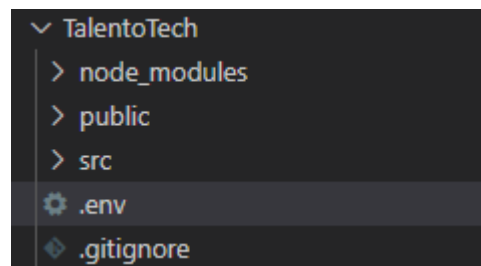
Pasos para el despliegue en Vercel:

1. **Abrir nuestra cuenta de GitHub:** Creemos un repositorio y subamos nuestro proyecto siguiendo los pasos que nos indica el repositorio. Subi solamente la carpeta del proyecto y recordá **no** subir el `.env` ni `node_modules` y dejar el proyecto como público.
2. **Crear una cuenta:** Registrarse en Vercel utilizando su cuenta de GitHub. Esto autorizará a Vercel a acceder a sus repositorios.
3. **Importar el Proyecto:** En el *dashboard* de Vercel, seleccionar "Add New... -> Project".
4. **Conectar el Repositorio:** Vercel mostrará una lista de sus repositorios de GitHub. Seleccionen el proyecto de nuestro eCommerce.
5. **Configurar el Despliegue:** Vercel detectará automáticamente que es un proyecto de React., sinó lo elegimos nosotros. Los comandos de *build* suelen ser correctos por defecto:
 - **Build Command:** `npm run build` o `vite build`.
 - **Output Directory:** `build` (para Create React App) o `dist` (para Vite)
 - **Install Command:** `npm install`.

6. **Variables de Entorno:** En la sección "Environment Variables", deben añadir las claves y valores que tengan en su archivo .env local.

Si no lo hiciste el .env en su momento, hagamoslo ahora. Crea en la raíz de tu proyecto un archivo .env (punto env)

En este archivo vamos a generar una variable y le asignaremos el valor de nuestra API KEY de ImgBB.



¡Importante! Como creamos el proyecto con Vite, este por seguridad, solo expone al cliente las variables de entorno que comienzan con el prefijo VITE_

ejemplo: **VITE_IMGGB_API_KEY=xx**

Para usar esta variable en nuestro archivo, vamos a llamarla de la siguiente manera de la siguiente manera:

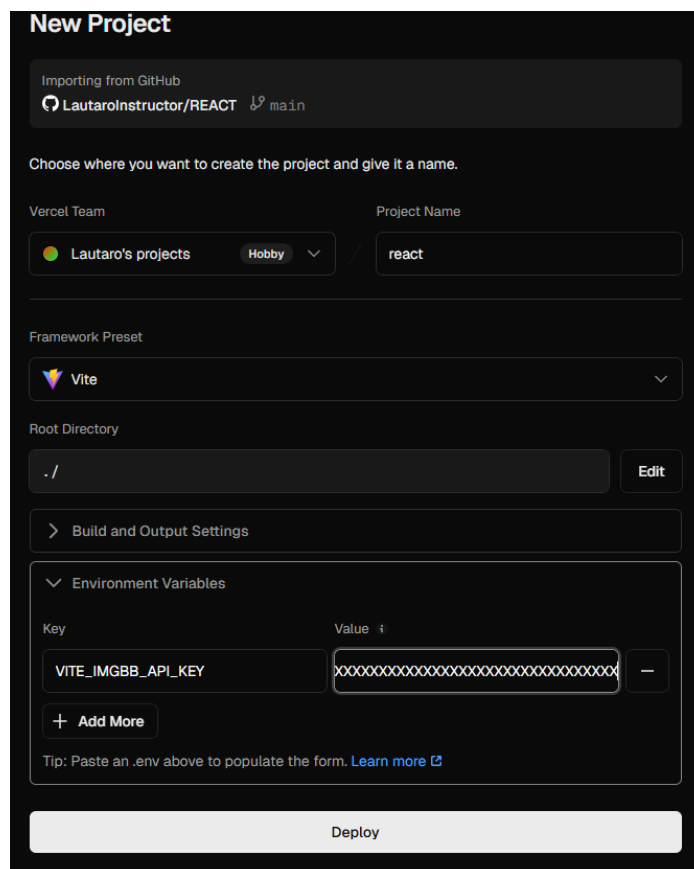
```
const apiKey = import.meta.env.VITE_IMGGB_API_KEY;
```

Probalo y si todo anda bien, continuemos.

Nuestra pantalla en Vercel tiene que quedar como se ve en la imagen:

7. **Desplegar:** Hacer clic en "Deploy". Vercel comenzará el proceso de instalación de dependencias, construcción del proyecto y despliegue.

Una vez finalizado, Vercel proporcionará una URL pública (ej: nuestro-ecommerce.vercel.app) donde la aplicación estará en línea y funcionando.



Desplegar en Netlify

Netlify es otra plataforma líder para el despliegue de sitios estáticos y aplicaciones web modernas, reconocida por su simplicidad y su robusto plan gratuito. Es muy similar a Vercel, por lo que vamos a arrancar de la misma manera:



1. Asegurate que las importaciones estén bien nombradas, teniendo en cuenta que es sensible a mayúsculas y minúsculas.
2. Borrá la carpeta node_modules
3. Borrá el archivo package-lock.json. **NO EL package.json.**
4. Probamos el desarrollo con **npm run build**.

Si no hay errores, pasamos al Deploy en Netlify

Pasos para el despliegue en Netlify:

1. **Abrir nuestra cuenta de GitHub:** Creemos un repositorio y subamos nuestro proyecto siguiendo los pasos que nos indica el repositorio. Subi solamente la carpeta del proyecto y recordá **no** subir el .env
1. **Registro en Netlify:** Registrate en Netlify y vincular la cuenta de GitHub
2. **Revisión de GitHub:** Asegúrense de que el repositorio esté actualizado.
3. **Crear un nuevo sitio desde Git:** En el dashboard de Netlify, seleccionar en la columna de la izquierda, "project". Luego "import from git". Elegis la cuenta de GitHub y si es la primera vez te pedirá que aceptes los permisos y a que repositorio/s vas a deployar.
4. **Configurar los parámetros de build:** Netlify también autodetectará la configuración. Verifiquen que los comandos sean los correctos.
 - **Build Command:** npm run build

Review configuration for REACT

Deploy as **LautaroInstructor** on **damaris** team from **main** branch using **npm** command and publishing to **dist**

Team
damaris

Project name
reactTalentTech4
https://reacttalenttech4.netlify.app
✔ Project name is available

Build settings
Specify how Netlify will build your project.
[Learn more in the docs](#)

Branch to deploy
main

Base directory
The directory where Netlify installs dependencies and runs your build command.

Build command
npm run build
Examples: jekyll build, gulp build, make all

Publish directory
dist
Examples: _site, dist, public

- **Public Directory:** build o dist
- 5. **Variables de entorno:** Asegurate de incorporar las variables de entorno en el deploy, justo debajo del publish directory de la imagen
- 6. **Desplegar:** Hacer clic en "Deploy site"

Al igual que Vercel, Netlify generará una URL aleatoria que se puede personalizar, donde la aplicación estará disponible que podés ver clickeando el mensaje de **published**.

Reflexión Final

Aprendimos a manejar el estado global de una aplicación React de una manera limpia y escalable utilizando la **Context API**. Esta herramienta es fundamental para evitar el *prop drilling* y nos permite desacoplar nuestros componentes, haciendo que nuestra base de código sea más mantenible y legible. Con la implementación del CartContext, hemos dado un paso crucial para construir una funcionalidad de eCommerce completa y robusta. Con nuestra página subida a internet, es momento de avanzar y profesionalizar nuestra aplicación.

Ejercicio práctico:

Cargar productos con fetch en ItemListContainer

Vamos a refactorizar nuestro listado de productos para que, en lugar de tener los datos "hardcodeados", los consuma de manera asíncrona desde un archivo local, simulando una llamada a una API real.

Modificar ItemListContainer.jsx:

1. Utiliza el hook `useState` para crear un estado `products`, inicializado como un array vacío.
2. Utiliza el hook `useEffect` para realizar una petición `fetch` al archivo `/productos.json` una sola vez, cuando el componente se monta.
3. Dentro del `.then()` de la promesa, actualiza el estado `products` con los datos recibidos del JSON.
4. Finalmente, en el `return` del componente, mapea el estado `products` para renderizar dinámicamente los componentes `Item`.

Consignas pre-entrega de proyecto final

Talento⁷ Lab



¡Hemos llegado a un momento clave! Es el momento de realizar la pre-entrega del proyecto trabajado hasta el momento. Como es de costumbre en varias empresas, nos toca pronto juntarnos con nuestro cliente para ver como va creciendo su sitio web. Esto nos va a permitir estar más en sintonía con las necesidades de nuestro cliente esperando su feedback y nos comentará por donde seguir. Por lo que venís trabajando te detallo lo que se espera tener esta primera presentación del sitio a nuestro cliente.

Objetivo: Integrar todas las habilidades aprendidas. El proyecto debe demostrar la correcta aplicación de componentes, manejo de estado, ruteo y estado global.

Requerimientos del proyecto:

A continuación se detalla el listado de funcionalidades que la aplicación debe cumplir, indicando la clase donde se aprendió el concepto principal.

1. Estructura y layout:

- El proyecto debe tener una estructura de carpetas organizada
- Debe contar con un componente Layout.jsx que contenga un Header.jsx , un nav y un Footer.jsx, con una apariencia consistente y funcional
- El footer tiene que tener información de la empresa y las tarjetas de al menos 3 personas.

2. Catálogo de productos con datos de una API:

- La aplicación debe tener un componente como ItemListContainer.jsx (o una página que cumpla esa función) que cargue la información de productos desde un archivo productos.json local usando useEffect y fetch.
- Los productos deben renderizarse utilizando un componente reutilizable Item.jsx, que reciba los datos por props.

3. Sistema de Ruteo:

- La navegación debe ser gestionada por react-router-dom.
- Deben existir, como mínimo, las siguientes rutas:

- /: Vista principal o de bienvenida.
 - /productos:
 - /producto/:id: Vista de detalle de un único producto
 - /carrito: Vista del carrito de compras.
 - El NavBar debe utilizar el componente <Link> para una navegación fluida sin recargas de página.
- 4. Funcionalidad del carrito con Context API:**
- Debe existir un componente que gestione el estado global del carrito
 - Desde la vista de detalle el usuario debe poder agregar productos al carrito. Esta acción debe llamar a la función addToCart del contexto.
 - El NavBar debe mostrar un ícono de carrito con un indicador numérico (CartWidget) que muestre la cantidad total de productos en el carrito. Este número debe obtenerse del CartContext y actualizarse en tiempo real.
 - La ruta /carrito debe mostrar el detalle de los productos agregados, consumiendo la información directamente del CartContext.
- 5. Alojamiento online:**
- La página debe estar subida a Netlify o Vercel y compartir la url, junto con el link al repositorio de GitHub.

Material y Recursos Adicionales:

- ★ [Documentación oficial de Context API](#)
 - ★ [Context API en tus proyectos](#)
-

Preguntas para Reflexionar:

- ¿Qué ventajas tiene usar Context API frente a pasar props entre componentes?
 - ¿Qué problemas podrías enfrentar al manejar múltiples contextos en una misma aplicación?
 - ¿Cómo podrías optimizar el rendimiento de una aplicación que usa Context API intensivamente?
-

Próximos Pasos:

- Refinar la experiencia de usuario en el carrito de compras.
- Implementar la renderización condicional para mostrar contenido dinámico.
- Ampliar la lógica de nuestro CartContext con nuevas funcionalidades.
- Creación de nuestro proyecto a Firebase



Buenos Aires
aprende

Agencia de Habilidades para el Futuro

BA Buenos
Aires
Ciudad